PRACTICAL 3: CLUSTERING FITNESS TRACKER DATA

Christoph Van Jaarsveld

STUDENT NUMBER: 50143956

Table of Contents

Project Overview

The objective of this practical assignment is to analyse simulated fitness tracker data by visualising the data on a scatter plot and applying the KMeans clustering algorithm to identify patterns within the dataset. The aim is to group users based on their activity levels and sleep patterns.

The simulated fitness tracker dataset consists out of three main activity level groups (clusters):

- **Cluster 1:** Active
- **Cluster 2**: Moderately Active
- **Cluster 3:** Least Active

Each cluster represents users with distinct levels of physical activity and sleep behaviour

For each of these three groups the following data attributes was recorded:

- **Steps per day**
- **Hours slept per Night**

Code Snippets

Libraries used

To plot and apply the KMeans algorithm on the data the following Python libraries were used in this assignment. NumPy was used for data generation and manipulation, Matplotlib for visualising the data with scatter plots and Scikit-learn to apply the KMeans clustering algorithm

```
# Christoph Van Jaarsveld 50143956
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
```

Generate Data

The following Python code generates the simulated fitness data dataset. The number of observations for each cluster is chosen randomly from the set of {100, 200, ..., 1000}. A fixed random seed is used in the report to ensure reproducible results

For each cluster, the total steps per day and total hours slept per night is generated at random within the following ranges:

Cluster	Steps per Day (Min)	Steps per Day (Max)	Hours Slept per Night (Min)	Hours Slept per Night (Max)
Cluster 1 (Active)	8000	12000	7	9
Cluster 2 (Moderately active)	5000	8000	6	7.5
Cluster 3 (Least active)	2000	5000	5	6.5

```
# Christoph Van Jaarsveld 50143956
# Set random seed for reproducibility
np.random.seed(42)
# Number of observations in each cluster - interval of 100 between 100 & 1000
c1 = np.random.randint(1, 11) * 100
c2 = np.random.randint(1, 11) * 100
c3 = np.random.randint(1, 11) * 100

# Total steps per day of each cluster
c1_steps = np.random.randint(8000, 12000, c1)
c2_steps = np.random.randint(5000, 8000, c2)
c3_steps = np.random.randint(2000, 5000, c3)

# Total hours slept per night of each cluster
c1_sleepH = np.random.uniform(7, 9, c1)
c2_sleepH = np.random.uniform(6, 7.5, c2)
c3_sleepH = np.random.uniform(5, 6.5, c3)

# Combine all clusters into a single dataset
x = np.concatenate((c1_steps, c2_steps, c3_steps))
y = np.concatenate((c1_sleepH, c2_sleepH, c3_sleepH))
data = np.column_stack([x, y])

print(f"\nTotal dataset size: {data.shape[0]} users")
```

Plot Data on Scatter Plot

The scatter plot x-axis and y-axis represents the Steps per Day and Hours Slept Per Night. The random generated cluster observations are as follows:

- **Cluster 1:** 700
- **Cluster 2:** 400
- **Cluster 3:** 800

The three clusters are easily distinguishable and are well separated into three blocks. There is minimal overlap between the three clusters since every active group falls within its own range. Cluster 1 and Cluster 3 appears denser than Cluster 2 because of the

higher number of data observations in Cluster 1 and Cluster 3. Cluster 1 is also more dispersed than the other two clusters because the number of steps per day and hours of sleep per night is higher compared to the other two clusters.

```
Total observations in cluster 1: 700
Total observations in cluster 2: 400
Total observations in cluster 3: 800
```

Figure 1: Generated Dataset Visualisation

```python
# Christoph Van Jaarsveld 50143956
# Print dataset information
print(f"Total observations in cluster 1: {c1}")
print(f"Total observations in cluster 2: {c2}")
print(f"Total observations in cluster 3: {c3}")

# Create labels for each cluster
labels = np.concatenate((
    np.full(c1, 0),
    np.full(c2, 1),
    np.full(c3, 2)
))

# Define cluster names and colors for the legend
cluster_info = {
    0: {"name": "Active", "color": "#1f77b4"},
    1: {"name": "Moderately active", "color": "#ff7f0e"},
    2: {"name": "Least active", "color": "#2ca02c"}
```

```
}

# Create the scatter plot
plt.figure(figsize=(10, 7))

# Iterate through cluster labels 0, 1, 2
for i in sorted(cluster_info.keys()):
    # Select data points for the current cluster
    cluster_x = x[labels == i]
    cluster_y = y[labels == i]
    plt.scatter(cluster_x, cluster_y,
                label=cluster_info[i]["name"],
                color=cluster_info[i]["color"],
                s=50,
                alpha=0.7)

# Add labels and title
plt.xlabel("Steps Per Day")
plt.ylabel("Hours Slept Per Night")
plt.title("Fitness Tracker Data Visualisation")
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```

Cluster Visualisation

After applying the KMeans clustering algorithm on the dataset, the data points were grouped into three clusters based on total steps taken per day and total hours slept per night. The KMeans clustering algorithm automatically assigned each user into a cluster based on the distance from each cluster centroid.

The resulting clusters shows a clear separation between activity groups:

- **Cluster 1 (Active):** Users with high activity and longer sleep .
- **Cluster 2 (Moderately Active):** Users with moderate activity and sleep.
- **Cluster 3 (Least Active):** Users with low activity and sleep

These results show that KMeans algorithm successfully identified the clusters that corresponds to each activity group.

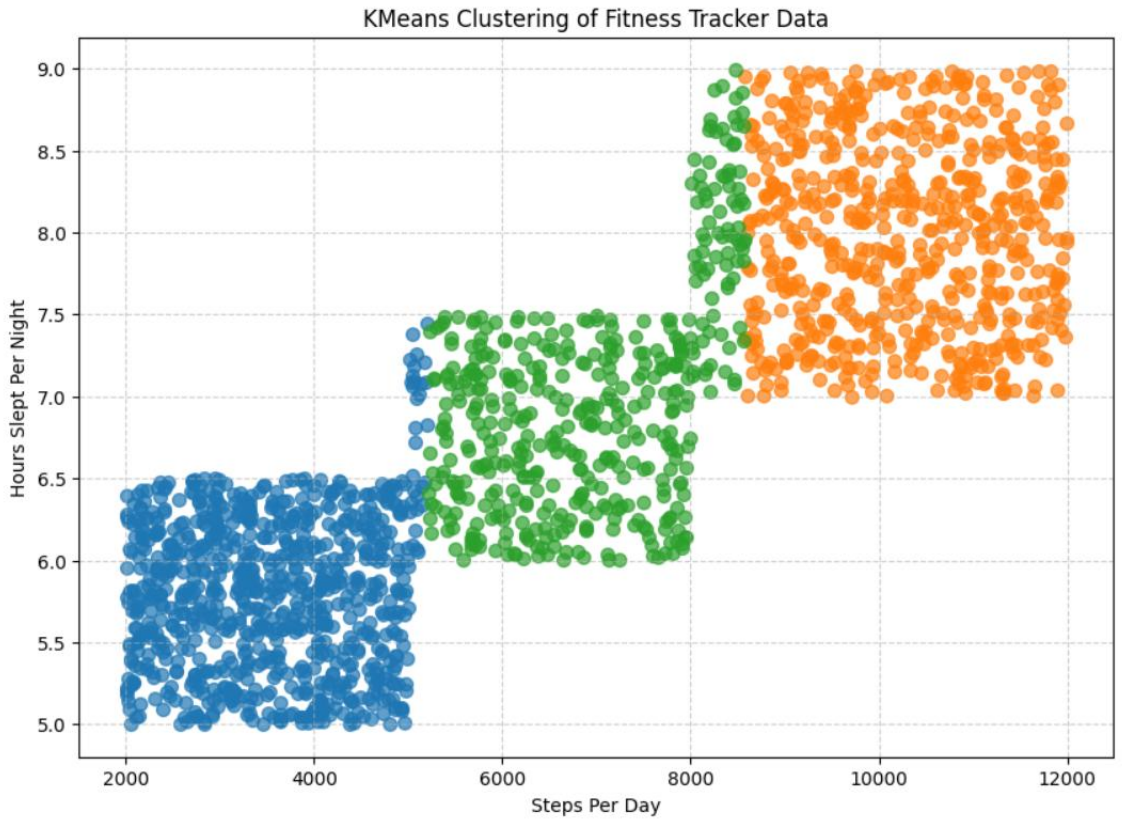

Figure 2: KMeans Clustering Visualisation

```
# Christoph Van Jaarsveld 50143956
# Number of clusters
n_clusters = 3

# Create a KMeans object with the specified number of clusters
kmeans = KMeans(n_clusters = n_clusters)

# Fit the KMeans model to the data
kmeans.fit(data)

# Get the cluster labels for each data point
kmeans_labels = kmeans.labels_

plt.figure(figsize=(10, 7))

# Iterate through the KMeans labels
for i in range(n_clusters):
    cluster_points = data[kmeans_labels == i]
    plt.scatter(cluster_points[:,0], cluster_points[:,1],
                s=50,
                alpha=0.7)
```

```
plt.xlabel("Steps Per Day")
plt.ylabel("Hours Slept Per Night")
plt.title("KMeans Clustering of Fitness Tracker Data")
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```

Cluster Analysis

By observing the cluster centroids, we can determine the average steps and hours slept for each group.

- **Centroid 1:** 3538 steps per day & 5.8 hours slept per night
- **Centroid 2:** 6919 steps per day & 7 hours slept per night
- **Centroid 3:** 10273 steps per day & 8 hours slept per night

Reviewing the centroid values, shows again that the KMeans clustering algorithm successfully identified clusters that correspond closely to the original activity groups

```
# Christoph Van Jaarsveld 50143956
# Analyse clusters
for i in range(3):

    cluster_points = data[kmeans_labels == i]

    avg_steps = round(np.mean(cluster_points[:,0]), 0) # average steps
    avg_sleep = round(np.mean(cluster_points[:,1]), 1) # average sleep

    print(f"\nCentroid {i+1}")
    print(f"Average Steps per Day: {int(avg_steps)}")
    print(f"Average Hours Slept: {avg_sleep}")
```

All Code Combined

```
# Christoph Van Jaarsveld 50143956
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans


# Set random seed for reproducibility
np.random.seed(42)
# Number of observations in each cluster - interval of 100 between 100 &
1000
c1 = np.random.randint(1, 11) * 100
c2 = np.random.randint(1, 11) * 100
```

```
c3 = np.random.randint(1, 11) * 100

# Total steps per day of each cluster
c1_steps = np.random.randint(8000, 12000, c1)
c2_steps = np.random.randint(5000, 8000, c2)
c3_steps = np.random.randint(2000, 5000, c3)

# Total hours slept per night of each cluster
c1_sleepH = np.random.uniform(7, 9, c1)
c2_sleepH = np.random.uniform(6, 7.5, c2)
c3_sleepH = np.random.uniform(5, 6.5, c3)

# Combine all clusters into a single dataset
x = np.concatenate((c1_steps, c2_steps, c3_steps))
y = np.concatenate((c1_sleepH, c2_sleepH, c3_sleepH))
data = np.column_stack([x, y])

print(f"\nTotal dataset size: {data.shape[0]} users")


# Print dataset information
print(f"Total observations in cluster 1: {c1}")
print(f"Total observations in cluster 2: {c2}")
print(f"Total observations in cluster 3: {c3}")

# Create labels for each cluster
labels = np.concatenate((
    np.full(c1, 0),
    np.full(c2, 1),
    np.full(c3, 2)
))

# Define cluster names and colors for the legend
cluster_info = {
    0: {"name": "Active", "color": "#1f77b4"},
    1: {"name": "Moderately active", "color": "#ff7f0e"},
    2: {"name": "Least active", "color": "#2ca02c"}
}

# Create the scatter plot
plt.figure(figsize=(10, 7))

# Iterate through cluster labels
for i in sorted(cluster_info.keys()):
    # Select data points for the current cluster
    cluster_x = x[labels == i]
    cluster_y = y[labels == i]
    plt.scatter(cluster_x, cluster_y,
                label=cluster_info[i]["name"],
                color=cluster_info[i]["color"],
                s=50,
                alpha=0.7)

# Add labels and title
plt.xlabel("Steps Per Day")
plt.ylabel("Hours Slept Per Night")
plt.title("Fitness Tracker Data Visualisation")
```

```
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()


# Number of clusters
n_clusters = 3

# Create a KMeans object with the specified number of clusters
kmeans = KMeans(n_clusters = n_clusters)

# Fit the KMeans model to the data
kmeans.fit(data)

# Get the cluster labels for each data point
kmeans_labels = kmeans.labels_


plt.figure(figsize=(10, 7))

# Iterate through the KMeans labels (0, 1, 2)
for i in range(n_clusters):
    cluster_points = data[kmeans_labels == i]
    plt.scatter(cluster_points[:,0], cluster_points[:,1],
                s=50,
                alpha=0.7)

plt.xlabel("Steps Per Day")
plt.ylabel("Hours Slept Per Night")
plt.title("KMeans Clustering of Fitness Tracker Data")
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()


# Analyse clusters
for i in range(3):

    cluster_points = data[kmeans_labels == i]

    avg_steps = round(np.mean(cluster_points[:,0]), 0) # average steps
    avg_sleep = round(np.mean(cluster_points[:,1]), 1) # average sleep
hours

    print(f"\nCentroid {i+1}")
    print(f"Average Steps per Day: {int(avg_steps)}")
    print(f"Average Hours Slept: {avg_sleep}")
```